Task 1

The code:

```python
###################
def substitute_chars(text, mapping):  1 usage
    table = str.maketrans(mapping)
    return text.translate(table)


ciphertext = open("ciphertext.txt").read()

# Mapping guesses: replace 'n' with 'E', 'y' with 'T', 'v' with 'A', etc.
char_mapping = {
    'y': 't','t': 'h','n': 'e','v': 'a','u': 'n','p': 'd',
    'm': 'i','r': 'g','q': 's','x': 'o','h': 'r','l': 'w',
    'a': 'c','z': 'u','g': 'b','d': 'y','e': 'p','b': 'f',
    'w': 'z','k': 'x','f': 'v', 's': 'k', 'c': 'm','i': 'l',
    'j': 'q','o': 'j'
}

partial_plaintext = substitute_chars(ciphertext, char_mapping)

# Save or print result
with open("plaintext.txt", "w") as f:
    f.write(partial_plaintext)
```

A sample of the plaintext:

```
the oscars turn  on sunday which seems about right after this long strange
awards trip the bagger feels like a nonagenarian too
```

NOTES:

The image shows Python code for decrypting a substitution cipher where each letter is replaced with another letter. It reads text from "ciphertext.txt", we applies character substitutions using a predefined mapping (like 'n' to 'e', 'y' to 't'), and saves the partially decrypted result to "plaintext.txt". The sample output reveals the text discusses the Oscars awards ceremony, though some words remain incorrectly decrypted ("bagger" instead of "winner"),indicating the mapping needs refinement for full accuracy because of some characters are to close in their frequency

TASK2:

This is the sample of the plaintext from task one:

```
the oscars turn  on sunday which seems about right after this long strange
awards trip the bagger feels like a nonagenarian too
the awards race was bookended by the demise of harvey weinstein at its outset
and the apparent implosion of his film company at the end and it was shaped by
the emergence of metoo times up blackgown politics armcandy activism and
a national conversation as brief and mad as a fever dream about whether there
ought to be a president winfrey the season didnt just seem extra long it was
extra long because the oscars were moved to the first weekend in march to
avoid conflicting with the closing ceremony of the winter olympics thanks
pyeongchang
```

AES-128-CBC Encryption:

```
serfayez@FayezHiary:~$ openssl enc -aes-128-cbc -e -in fayez.txt -out cipher1.bin \
-K 00112233445566778899aabbccddeeff \
-iv 000102030405060708090a0b0c0d0e0f
serfayez@FayezHiary:~$ ls
cipher1.bin  fayez.txt
```

```
1ù▯ò▯▯D▯xÝš8öÁõ‹▯LªGÂžœj©cËï▯«—▯Á=†õÕ¬óêæ ºŸ5*åÓhª“ê—h ‰Ð@
ÂPcÔÝy▯ä        /¤—£Úóc×·¨▯J_5Ù9ÎJ▯9▯▯Rõ▯‰i}÷oåÊ»ñå;Ú ▯ÆÔE0¬ä¤Ó˜Ÿ▯E]▯@_É—€÷=▯2)
v▯@¯▯1▯Ç▯ñö¸ôý41ƒ▯

Ø¹ä/g{ö”Íå▯EV▯ˆ6¬„²1•N5¶?ÙB09"þ*IŠ…*ì▯ÆŠ¸çÓ6 zÒý5▯ZFéû º▯<‰É^9▯_ÔÄÖ n$s}▯
Ð▯Zì\—▯'¼<▯•àwPšQwÎ¯™▯▯
r\▯/œwÊ›´n½”▯▯“ŸÀ³ Sƒ5s▯âA▯Çd?m×û£¥1ôä3I”yÓ¨▯Q6ænJùlª,Û%▯TO¤Æ! ¿§û 3zÕÒ@ex `
$Æk▯“g#Úµ▯$tÌ º8▯çë„}r)Ê›XEeõ6*nUÔÊ▯ê▯È’▯▯ì▯‹.ŠÛsíÇNq>0Ó=á
▯ö1ëüŒãÃœ€ìØ¸ZRr ‘VGf]tª0▯˜I!▯ž=’▯2Ç▯™Y▯ÀÙHå—Ž8▯×X▯▯ÛBÔ[|Zqâ1▯âd▯›▯8¯Cj
˜Î&▯Eú}±vœê{£ã¨?°ððánóÅbd▯£žD“$ã▯▯pƒÝyYÇîoëÞ¸“]2‹7▯Ç…æç▯Ó¸†¾▯P4R¹"%
ÔqÙ{Åý§â±▯åßšJf±%q▯BSå®è¼£·▯NUQLLØŽI©áÈÇ:ïê>Y-‹ØM▯ˆ>ªÇk/\í£Z#2x˜ð▯AÓªG¿½-
OÑ>5ò+d…Ÿ▯▯▯Û¨¾%‹ø°P2ÌòŒüêªnb-z IÜ`æ©íIoNÍ▯4í"  ¨lu£[—,(▯EQÈ:ç…Ž*D3▯Ø▯§
]▯ó¿î▯hÍp▯▯Ë6£OZà▯‘_¥▯▯‘£ûÒô7m▯
```

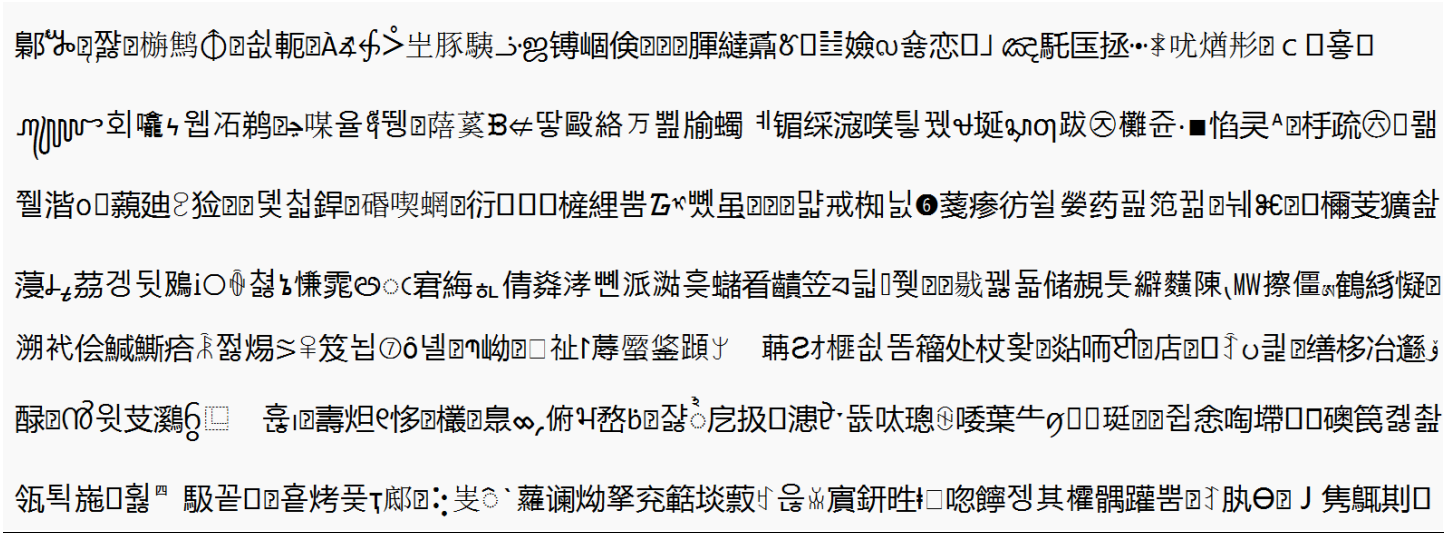
AES-128-CFB Encryption:

```
serfayez@FayezHiary:~$ openssl enc -aes-128-cfb -e -in fayez.txt -out cipher2.bin \
-K 00112233445566778899aabbccddeeff \
-iv 000102030405060708090a0b0c0d0e0f
serfayez@FayezHiary:~$
```

```
S÷Òj▯▯p?ýè®,¤œŽ#ŸëLa▯O^ïör¡Zþ×‹˜—▯OW·§„(qô`▯á©ó+IÍk`ÍH˜éTl▯·9+%▯
û"µ▯Lgé)D-!▯▯ÌðD›Ä“7·▯Ø/¬.úy▯`]¤›aÄò©▯å4ó Â|S▯s’▯▯MƒÿÜ>e“˜ªáœàúÝoæ•¹êg 'x▯Œ#▯e¿”
Ôt▯▯”Ã¿k
ß{rpýÏ&ýþmêû»X’ž¶†ÂÆ÷Ã<|÷Y▯Äï^|¸àme•ì▯▯õ…%]e%¹þµÕA▯s8lí8ž=•ç¯4▯V‰▯ªz▯ó0{
        aT▯=A'ÍL—>á+[í¥D▯àÍ°8Ý”N▯¾Â=…Ú▯▯R¾
▯p4m_?+’▯n„ke¦¨{)d²▯ûö0ö<é[{úœm/ÀHâTbÑ▯ê]4Ýò†\ ¥▯•Ÿ’²í-Q¿zŽw   ‰%mi=iL[~’▯ù
Ì5Pì‹R“íéðÓ½ÌÎvñ”▯œ0~ÈÙk\▯&å©Š▯!▯ÚÈéó
7Z©tµ;ó&Ÿxá×´¤▯i¦<▯»Ï¢ÏÝ«ÛÏ1ƒ>S▯uu▯▯˜³▯±;‡^± b¨·ºC@³îÝ{ì▯k¹'Ž1▯u°▯y—È»?{Õìøßï
€▯ž×▯▯*"{ž▯{aµ½D™▯Qà▯º▯:˜
ìá¾µT€▯▯Ø▯ã▯¸▯ÈU}º¶òÅÎÁ>ç▯o /▯+2å´¤▯üP▯▯§°ê▯˜G™¤Ê8\ë4¥†Ò©JÔ£    ▯XNÍg¶ç*p=T¢
8ÌÉ"Ñ▯ðJH1ŠPÓ0▯▯Óð\C‹çžµaP@}} ,è¹,+˜Ž}(▯U—º~c▯▯▯        EPêBW¾ò¾8šOÎÙ<bEË ▯
\’▯¯Vxþc¿L.G'˜:µ▯<{f▯}¿~S®z▯▯)JŠÝ`üúMô
l›ùÞ©Ô\▯A0‹1y´´Ä•G†
```

BF-CBC Encryption:

```
serfayez@FayezHiary:~$ export OPENSSL_CONF=/etc/ssl/openssl.cnf
serfayez@FayezHiary:~$ openssl enc -bf-cbc -e -in fayez.txt -out cipher3.bin
 \
-K 00112233445566778899aabbccddeeff \
-iv 0102030405060708 \
-provider legacy -provider default
serfayez@FayezHiary:~$
```



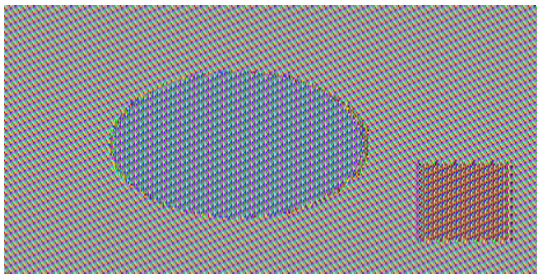
TASK3

Encrypt using ECB and CBC:

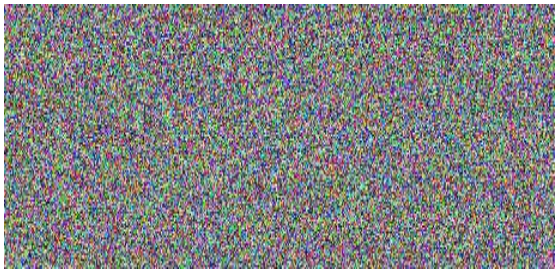
```
serfayez@FayezHiary:~$ cp "/mnt/c/Users/User/OneDrive/Desktop/pic_original.bmp" ~/original.jpg
```

```
serfayez@FayezHiary:~$ convert ~/original.jpg ~/original.bmp
serfayez@FayezHiary:~$ tail -c +55 ~/original.bmp > ~/data_only
serfayez@FayezHiary:~$ sudo apt install openssl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
openssl is already the newest version (3.0.13-0ubuntu3.5).
openssl set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 97 not upgraded.
serfayez@FayezHiary:~$ openssl enc -aes-128-ecb -in ~/data_only -out ~/encrypted_ecb -K 00112233445566778889aabbccd
deeff -nosalt -nopad
hex string is too long, ignoring excess
serfayez@FayezHiary:~$ openssl enc -aes-128-cbc -in ~/data_only -out ~/encrypted_cbc -K 00112233445566778889aabbccd
deeff -iv 0102030405060708090a0b0c0d0e0f10 -nosalt -nopad
hex string is too long, ignoring excess
serfayez@FayezHiary:~$ head -c 54 ~/original.bmp > ~/header
serfayez@FayezHiary:~$ cat header encrypted_ecb > encrypted_ecb.bmp
serfayez@FayezHiary:~$ cat header encrypted_cbc > encrypted_cbc.bmp
serfayez@FayezHiary:~$ eog encrypted_ecb.bmp
eog encrypted_cbc.bmp
```

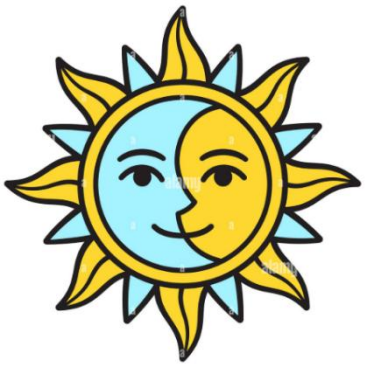
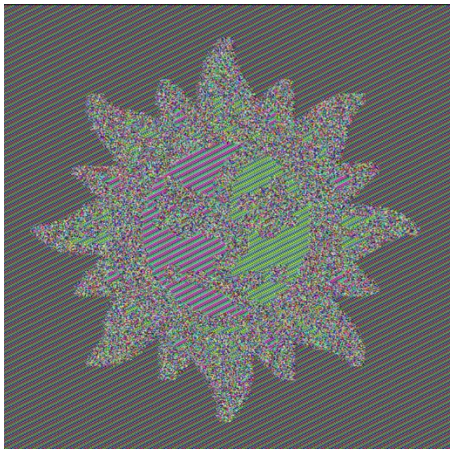
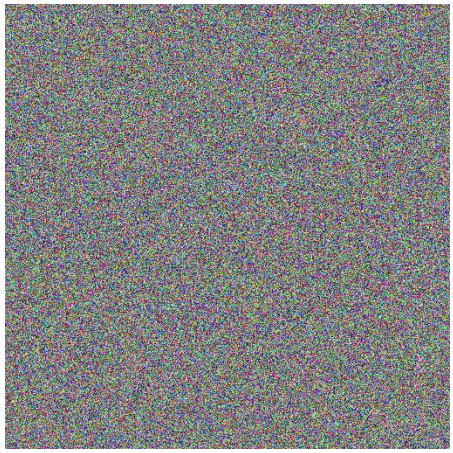
ECB:



CBC:



Repeated Experiment with image of our choice:

Explanation: ECB is not secure for image encryption because it leaks visual information (if you use same plaintext and same key produce same ciphertext). You can't see exact colors or details, but the outline and structure of the image are still visible.

CBC is much better at hiding information in image encryption(chaining method solved ECB problem). The original picture cannot be visually reconstructed.

TASK4

1) The reason ECB and CBC are padded, while CFB and OFB are not padded has to do with stream vs block ciphers. Since ECB and CBC are block ciphers, files must be padded until a multiple of the block size is reached. CFB and OFB are stream ciphers, so they do not need to be padded.

2)

```
serfayez@FayezHiary:/mnt/c/Users/User$ echo -n "12345" > A1.txt
serfayez@FayezHiary:/mnt/c/Users/User$ echo -n "1234567890" > A2.txt
serfayez@FayezHiary:/mnt/c/Users/User$ echo -n "1234567890ABCDEF" > A3.txt
serfayez@FayezHiary:/mnt/c/Users/User$ KEY="00112233445566778899aabbccddeeff"
serfayez@FayezHiary:/mnt/c/Users/User$ IV="0102030405060708090a0b0c0d0e0f10"
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -e -in A1.txt -out A1.enc -K $KEY -iv $IV
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -e -in A2.txt -out A2.enc -K $KEY -iv $IV
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -e -in A3.txt -out A3.enc -K $KEY -iv $IV
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -d -in A1.enc -out A1.dec -K $KEY -iv $IV -nopad
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -d -in A2.enc -out A2.dec -K $KEY -iv $IV -nopad
serfayez@FayezHiary:/mnt/c/Users/User$ openssl enc -aes-128-cbc -d -in A3.enc -out A3.dec -K $KEY -iv $IV -nopad
serfayez@FayezHiary:/mnt/c/Users/User$ xxd A1.dec
00000000: 3132 3334 350b 0b0b 0b0b 0b0b 0b0b 0b0b  12345...........
serfayez@FayezHiary:/mnt/c/Users/User$ xxd A2.dec
00000000: 3132 3334 3536 3738 3930 0606 0606 0606  1234567890......
serfayez@FayezHiary:/mnt/c/Users/User$ xxd A3.dec
00000000: 3132 3334 3536 3738 3930 4142 4344 4546  1234567890ABCDEF
00000010: 1010 1010 1010 1010 1010 1010 1010 1010  ................
serfayez@FayezHiary:/mnt/c/Users/User$
```

As you see A1 is 5-Bytes (In Green), and the padding of A1 is 11-Bytes (In Red) [Padding of 11 bytes (0x0B) added to fill the 16-byte block]

A2 is 10-Bytes (In Green), and the padding of A1 is 6-Bytes (In Red) [Padding of 6 bytes (0x06) added to complete 16-byte block]

A3 [Even though this is exactly one block, adds an entire block of padding: 16 bytes]

TASK5

ECB: You lose 1 block, All other blocks remain intact, so you can recover most of the plaintext except for the corrupted block.
CBC: You lose 2 blocks (corrupted one and the next), but the rest of the file is recoverable.
CFB: You lose 2 blocks, then recovery resumes. So most of the file is still recoverable, with minor corruption.
OFB: Only the corrupted part is lost, all other data is recoverable.

1)

Sample of plaintext:

```
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
```

TASK6

6.1)

```
serfayez@FayezHiary:~$ KEY=00112233445566778889aabbccddeeff
IV1=00000000000000000000000000000000
IV2=01010101010101010101010101010101
serfayez@FayezHiary:~$ openssl enc -aes-128-cbc -K $KEY -iv $IV1 -in FF.txt -out enc_iv1.bin
serfayez@FayezHiary:~$ openssl enc -aes-128-cbc -K $KEY -iv $IV2 -in FF.txt -out enc_iv2.bin
serfayez@FayezHiary:~$ openssl enc -aes-128-cbc -K $KEY -iv $IV1 -in FF.txt -out enc_iv1_repeat.bin
```

Sample of plaintext:

```
the oscars turn  on sunday which seems about right after this long strange
awards trip the bagger feels like a nonagenarian too

the awards race was bookended by the demise of harvey weinstein at its outset
and the apparent implosion of his film company at the end and it was shaped by
the emergence of metoo times up blackgown politics armcandy activism and
```

Encryption with IV1:

```
^¢ŸjÌ¸Ê⍰A-º7\Pb^‡Œ\@¾L\Â9
ÎYF!å¸²⍰ÚnE÷Nw
AZÅA×æÖŸîâLŽ¶\p‰u,c⍰X4ß©:-§ªC⍰⍰æ8}û⍰Z×l>C£k€õ
        Œ{⍰⍰®uk·Ê⍰Èå⍰=Z>„þdbå⍰Ä⍰2⍰⍰Ï*
è%¶á⍰)eüÔó(ŠBÜ×ZR⍰«‘Cl⍰^ÞæM¿<š|ir‹“⍰R„⍰⍰{¾⍰
LëknkqÛˆJHc4ï©ƒÞ‡Ù„½×.§8€;Ë⍰⍰Ry⍰⍰Ê⍰v⍰¥¾Íl"‰MÚ⍰Ó„/p⍰`˜Î⍰Äö6.¦µ‰  !5åÁ>
```

Encryption with IV2:

```
謙ᡒ뱇虐破ᡐ娶赮ᢀ厖뾣□◯轎ᡒ⍰碻忼□苯獠⍰。⍰奚⍰縰⍰笠ᡒ翎缋ク稼欶箇稱ᡒ栖惴崮ᡒ楸辩ᡐᢀ⍰⍰⍰隱⍰鶤ᡐ溺ᡒ⍰ᡒ⍰鈍

ᡒ⍰⍰瀨伝⍰ᡐ夐퍾ᡒ뼩顎齡◊□搥ᡐ⍰ᡒ輬ᡒ湉돦捷邦곲ᡒ퓟ᡒ毀ᡒ匡笠ᡐ⍰瑊ᡒ儿胠⍰⍰ᡒ齋⍰ᡒ蘿磝ᡒ蚪娟铆헦檚矜삭ᡐᡒ⍰ᡒ：ᡐ

□車欽ᡒ鏶鞲□ᡒᡐ墨ᡐ⍰竇ᡒ棍ᡐ□躓ᡐ趲鬻蝻궷　湣榡穰錼⍰賴跕达⍰□嫁袂ᡐ⍰⍰臭ᡐᡒ痦戶硎鞯魎嵊夯但⍰맞硪萦⍰蚕璇
```

Encryption with IV1 Again:

```
^¢ŸjÌ¸Ê⍰A-º7\Pb^‡Œ\@¾L\Â9
ÎYF!å¸²⍰ÚnE÷Nw
AZÅA×æÖŸîâLŽ¶\p‰u,c⍰X4ß©:-§ªC⍰⍰æ8}û⍰Z×l>C£k€õ
        Œ{⍰⍰®uk·Ê⍰Èå⍰=Z>„þdbå⍰Ä⍰2⍰⍰Ï*
è%¶á⍰)eüÔó(ŠBÜ×ZR⍰«‘Cl⍰^ÞæM¿<š|ir‹“⍰R„⍰⍰{¾⍰
LëknkqÛˆJHc4ï©ƒÞ‡Ù„½×.§8€;Ë⍰⍰Ry⍰⍰Ê⍰v⍰¥¾Íl"‰MÚ⍰Ó„/p⍰`˜Î⍰Äö6.¦µ‰  !5åÁ>
```

Explanation: IV needs to be different every time. If the same IV is used every time, it will become easy for an attacker to analyze the pattern whenever a plaintext is reused

6.2)

sample_code.py

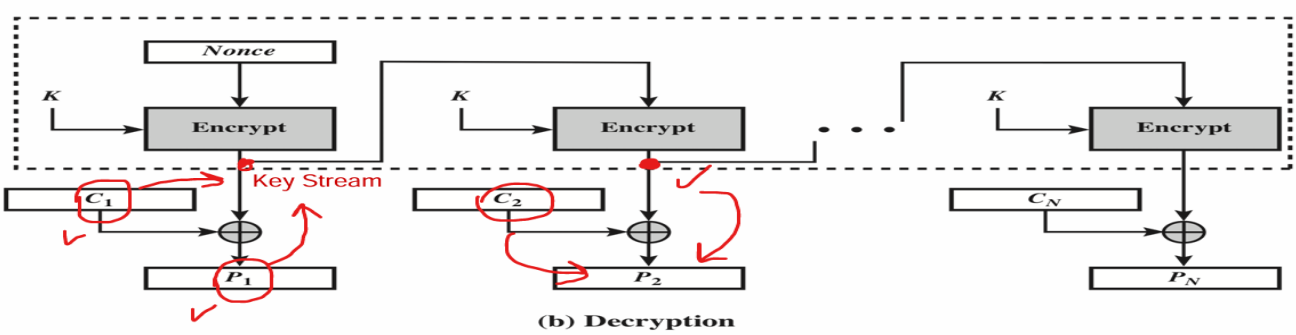
```python
#!/usr/bin/python3
# XOR two bytearrays
def xor(first, second):  2 usages
    return bytearray(x ^ y for x, y in zip(first, second))
# Given data
P1 = "This is a known message!"
C1_hex = "a469b1c502c1cab966965e50425438e1bb1b5f9037a4c159"
C2_hex = "bf73bcd3509299d566c35b5d450337e1bb175f903fafc159"
# Step 1: Convert ascii plaintext to bytearray
D1 = bytes(P1, 'utf-8') # plaintext
# Step 2: Convert ciphertext hex to bytearrays
D2 = bytearray.fromhex(C1_hex) #C1
D3 = bytearray.fromhex(C2_hex) #C2
# Step 3: Get the keystream by XORing plaintext and C1
keystream = xor(D1, D2) # key stream = encrypted Nonce
# Step 4: Get P2 by XORing C2 and keystream
P2_bytes = xor(D3, keystream)
# Output the recovered P2
print(P2_bytes.decode('utf-8'))
```

sample_code

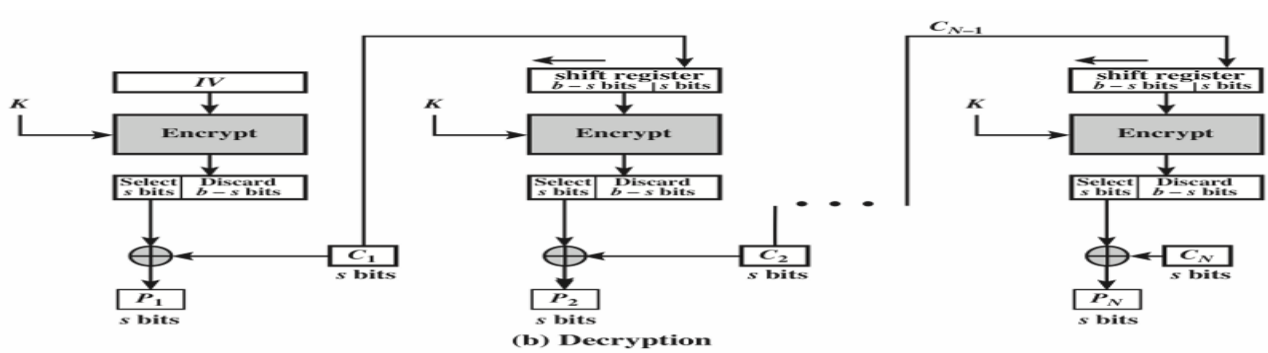
```
C:\Users\User\anaconda3\python.exe C:\Users\User\OneDrive\Desktop\Labsetup\Files\sample_code.py
Order: Launch a missile!

Process finished with exit code 0
```

To be clear:



If this experiment is done using CFB instead of OFB, we can only decipher the first block of the message(the OFB not secure against known plaintext , I can know the whole message .But CFB we consider it secure because we can only know the first block against Known plaintext)



6.3) Bob sends YES or No as plaintext encrypted

Eve knows Ciphertext(C1) and IV And the next IV (Because Bob uses IV Predictable)

Eve enters plaintext (Yes or no, guessing) to Bob and Bob encrypt this plaintext using IV (Belong to Bob) that are already known by Eve and gives C2. (Eve here is a chosen plaintext)

The weakness here is that Eve knows the IV.

Now Eve by guessing (for example she entered Yes(P2)) knows she will compare Between C1 and C2 if they are match then the P1 is "Yes"

If not, then P1 is "NO".

summary:

Since I know that the plaintext is either “yes” or “no”, I just need to guess one of the two and then predict the IV from the ciphertext. Then, I can verify my guess by encrypting my plaintext guess with the known IV and comparing the result.